

Full Stack Application – Part 2



04

Every online interaction, from signing up on a website to enrolling in a course, relies on a powerful backend that manages data and processes requests seamlessly.



Modern web applications rely on a structured system to function smoothly. When users interact with a website, their data is processed, stored, and retrieved at the backend.

Consider an online food delivery app. When you place an order, the app does not just display your selection; it also processes the payment, updates the restaurant, and manages delivery. Similarly, in our course enrollment application, the backend will handle data storage, retrieval, and processing, working behind the scenes to support the frontend.

In the previous lesson, we built the frontend using React. Now, we will focus on backend development. We will use MySQL to create a database and tables for storing user input. Node.js will act as a bridge between the frontend and the database, ensuring smooth interaction.

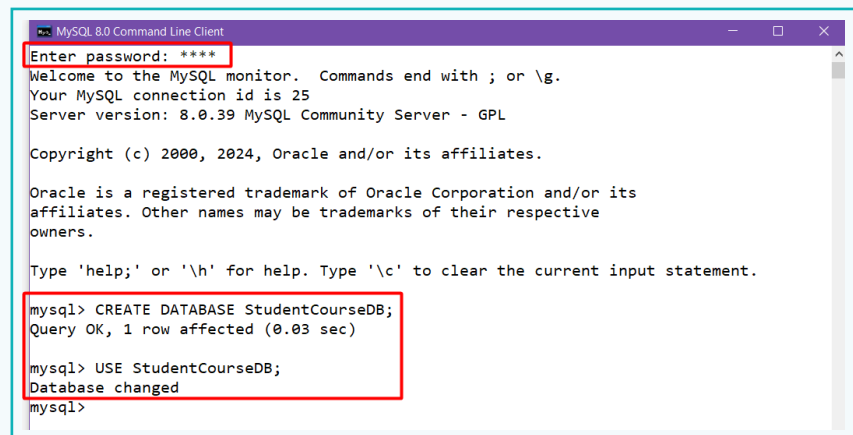
Let us begin by setting up the database and tables in MySQL. Before proceeding, ensure that MySQL is installed on your system and that you are familiar with its basics.

Activity

Part 1 - Create Database and Tables

First, we will set up a database called 'StudentCourseDB' and create two tables within it: 'Courses' and 'Students'.

- 1 Open the MySQL database client and log in.
 - Next create a database 'StudentCourseDB' using the *CREATE DATABASE* command and select it using the *USE* command.



```
MySQL 8.0 Command Line Client
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 25
Server version: 8.0.39 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

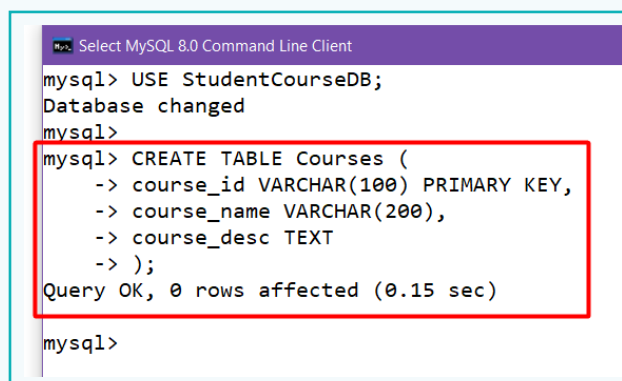
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE StudentCourseDB;
Query OK, 1 row affected (0.03 sec)

mysql> USE StudentCourseDB;
Database changed

mysql>
```

- 2 Create a table 'Courses' as shown. Make 'course_id' field as the primary key.



```
Select MySQL 8.0 Command Line Client

mysql> USE StudentCourseDB;
Database changed

mysql>
mysql> CREATE TABLE Courses (
->   course_id VARCHAR(100) PRIMARY KEY,
->   course_name VARCHAR(200),
->   course_desc TEXT
-> );
Query OK, 0 rows affected (0.15 sec)

mysql>
```

Activity

- Refer the below table and insert new records in the 'Courses' table.

Course_id	Course_name	Course_desc
WEB-101	Learning HTML and CSS	A beginner - level course that teaches the basics of HTML and CSS to build websites.
CS-102	Getting Started with Computer Science	An introductory course that focuses on developing the fundamentals of Computer Science.
DS-103	Data Science for Beginners	This course introduces the basics of Data Science, including data manipulation and visualization.

```
MySQL 8.0 Command Line Client
Query OK, 0 rows affected (0.15 sec)

mysql> INSERT INTO Courses VALUES
    -> ('WEB-101', 'Learning HTML and CSS', 'A beginner level course that teaches
the basics of HTML and CSS to build websites');
Query OK, 1 row affected (0.03 sec)

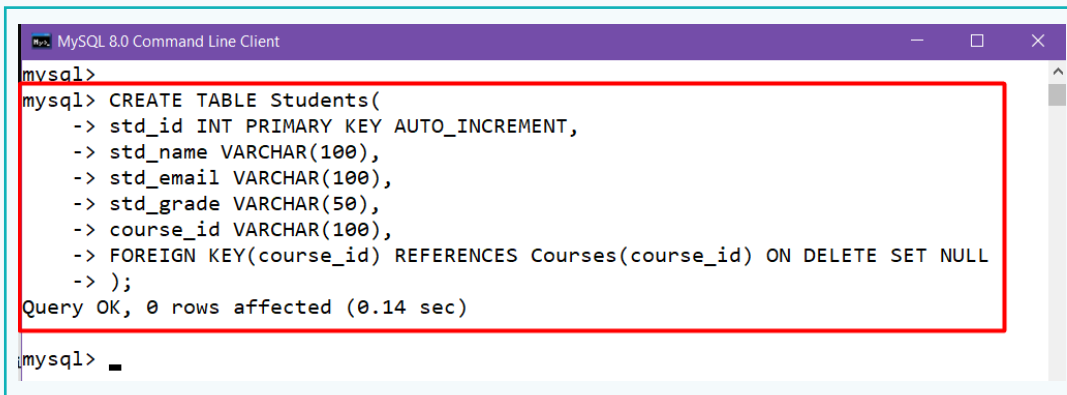
mysql> INSERT INTO Courses VALUES
    -> ('CS-102', 'Getting Started with Computer Science', 'An introductory course
that focusses on developing the fundamentals of Computer Science');
Query OK, 1 row affected (0.02 sec)

mysql> INSERT INTO Courses VALUES
    -> ('DS-103', 'Data Science for Beginners', 'This course introduces the basics
of Data Science, including data manipulation and visualization');
Query OK, 1 row affected (0.02 sec)

mysql>
```

Activity

- 3 Next, create a 'Students' table with 'std_id' as the primary key.
 - Declare this field as auto-increment using the *AUTO_INCREMENT* property. This will automatically generate a unique value for the 'std_id' field whenever a new record is inserted into the table.



```
mysql>
mysql> CREATE TABLE Students(
-> std_id INT PRIMARY KEY AUTO_INCREMENT,
-> std_name VARCHAR(100),
-> std_email VARCHAR(100),
-> std_grade VARCHAR(50),
-> course_id VARCHAR(100),
-> FOREIGN KEY(course_id) REFERENCES Courses(course_id) ON DELETE SET NULL
-> );
Query OK, 0 rows affected (0.14 sec)

mysql>
```

In the 'Students' table, we have declared 'course_id' as a Foreign Key and made a reference to the 'course_id' field in the 'Courses' table. *ON DELETE SET NULL* constraint ensures that if the referenced 'course_id' is deleted in the 'Courses' table, the 'course_id' field in the 'Students' table will be set to NULL.

Note:

We are not inserting any records into the 'Students' table at this stage. Instead, records will be inserted later through the React application (frontend). To achieve this, we must create a Node.js project that will enable dynamic data storage and retrieval.

Node.js

Node.js is a JavaScript runtime environment that allows you to execute JavaScript code on the server side. It is widely used for building backend applications, handling APIs, and managing databases in full-stack development. In our application, Node.js will act as a bridge between the React frontend and the MySQL database. For example, when a user submits the 'StudentForm' component in React, Node.js will store that data in the 'Students' table.

Now, let us create a Node.js project and implement this functionality. Recall that we created the 'backend' folder in Part 1 (previous lesson). We will now set up the backend within that folder.

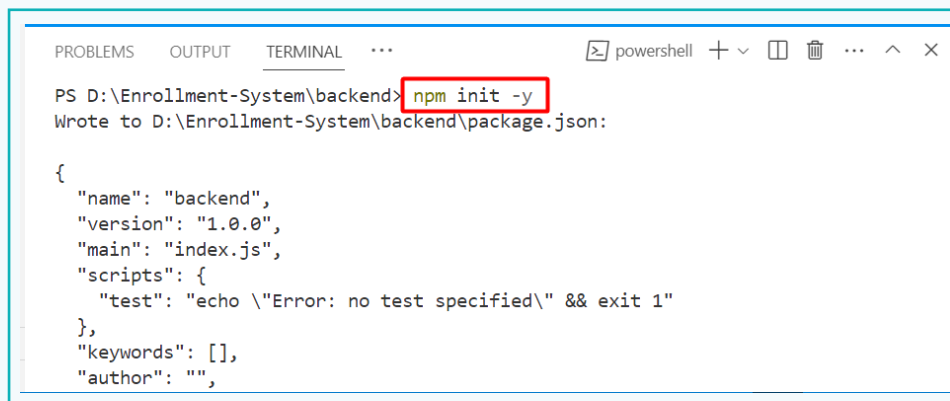
Activity

Part 2 - Create a Node.js Project

Note:

Ensure that Node.js is installed on your system before proceeding with the next steps.

- 1 Open the 'backend' folder in VS Code. In the terminal, execute the command `npm init -y` to create a Node.js project.



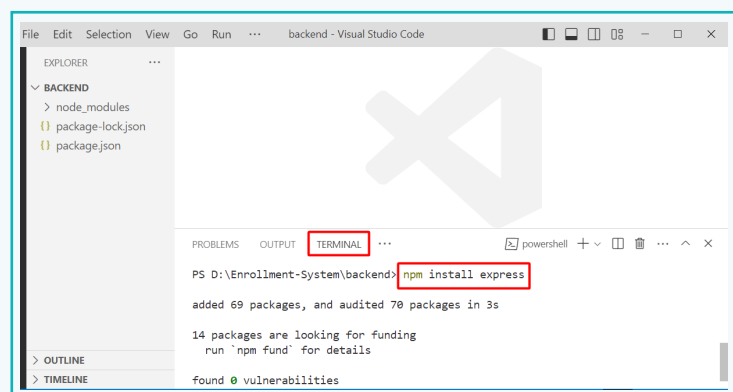
```
PS D:\Enrollment-System\backend> npm init -y
Wrote to D:\Enrollment-System\backend\package.json:

{
  "name": "backend",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
```

Next, we will install the Express library. **Express** is a web framework for Node.js that simplifies the process of building server-side web applications.

Part 3 - Install the Express Library

- 1 In the terminal, execute the command `npm install express` to install the Express Framework.



```
File Edit Selection View Go Run ... backend - Visual Studio Code
EXPLORER
  BACKEND
    > node_modules
    {} package-lock.json
    {} package.json

PROBLEMS OUTPUT TERMINAL ...
PS D:\Enrollment-System\backend> npm install express
added 69 packages, and audited 70 packages in 3s

14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Now that we have installed Express, let us set up a web server to handle requests from our React application.

Activity

Web Server: A web server acts as a bridge between the frontend and the database, ensuring smooth data exchange. It processes incoming requests from clients (such as web browsers) and responds with the requested data or webpage.

In this application, we will create a basic web server using Express. This server will handle and process requests from our React application, enabling seamless communication between the frontend and the database.

Create a Web Server

2 First, create a file named 'index.js', which will serve as the main entry point for our application. In this file, we will write code to set up a web server using Express.

- Import the Express framework, using the *require* statement and then create its object using the *express()* method.
- Use the *app.listen()* method to start the web server and make it listen for incoming requests on port 5000. Inside this method, display the statement "Server is Running" using *console.log()* to indicate the server has started and is running.

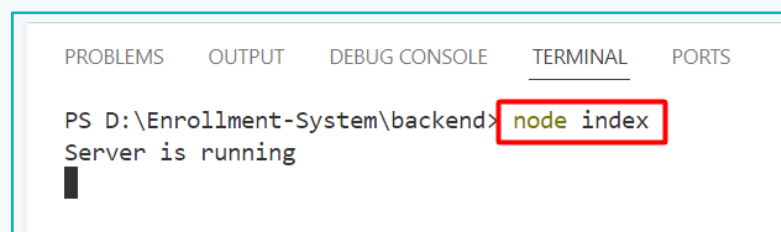


The screenshot shows the VS Code interface. On the left, the Explorer sidebar shows the 'BACKEND' folder containing 'node_modules', 'package-lock.json', 'package.json', and 'index.js'. The 'index.js' file is selected and highlighted with a red box. On the right, the Editor shows the code for 'index.js' with line numbers 1 through 8. The code is as follows:

```
1 const express = require('express');
2 const app = express();
3
4 app.listen(5000, () => {
5   console.log('Server is running');
6 });
7
8
```

The code block is highlighted with a red box.

- Execute the application by typing the command *node index* on the terminal.



The screenshot shows the VS Code interface with the 'TERMINAL' tab selected. The terminal shows the command 'node index' being executed in the directory 'D:\Enrollment-System\backend'. The output is 'Server is running'.

```
PS D:\Enrollment-System\backend> node index
Server is running
```

The command 'node index' is highlighted with a red box.

Our server has started. Next, we will learn how to connect our Node.js project to the 'StudentCourseDB' database.

Activity

To simplify the process of connecting a Node.js project with a MySQL database, we need to install a separate package called **mysql2**. This Node.js package allows us to connect and operate a MySQL database using basic codes.

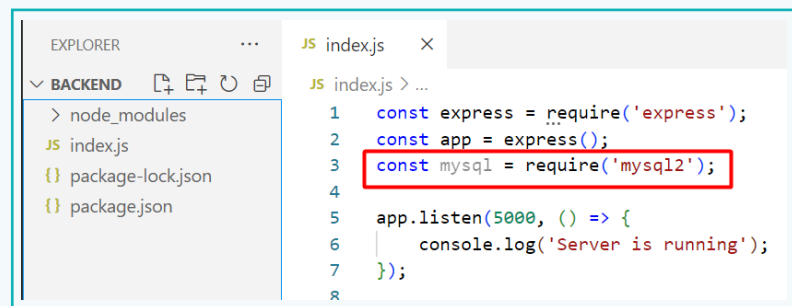
Part 4 - Connect to MySQL Database

- 1 First, we need to install the package. Navigate to the terminal and execute the command `npm install mysql2`.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\Enrollment-System\backend> npm install mysql2
```

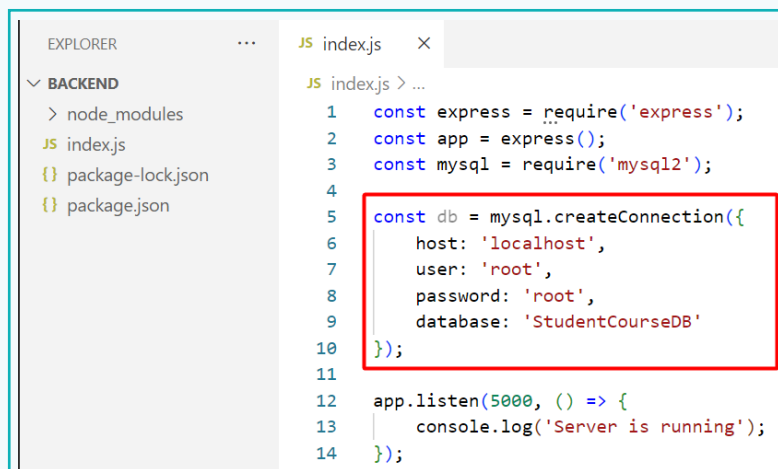
- 2 Now, we need to import the `mysql2` package to integrate it into our project.



```
EXPLORER  JS index.js  X
  BACKEND  JS index.js > ...
    > node_modules
    JS index.js
    {} package-lock.json
    {} package.json

1  const express = require('express');
2  const app = express();
3  const mysql = require('mysql2');
4
5  app.listen(5000, () => {
6    console.log('Server is running');
7  });
8
```

- 3 Next, create a connection object using the `mysql2` package, as shown.



```
EXPLORER  JS index.js  X
  BACKEND  JS index.js > ...
    > node_modules
    JS index.js
    {} package-lock.json
    {} package.json

1  const express = require('express');
2  const app = express();
3  const mysql = require('mysql2');
4
5  const db = mysql.createConnection({
6    host: 'localhost',
7    user: 'root',
8    password: 'root',
9    database: 'StudentCourseDB'
10 });
11
12 app.listen(5000, () => {
13   console.log('Server is running');
14 });
```

Activity

Here,

- **host: 'localhost'**- Connects to the local MySQL server.
- **user: 'root'**- Uses the MySQL root user.
- **password: 'root'**- Password for the root user.
- **database: 'StudentCourseDB'**- Specifies the database to connect to.

The code above creates a connection object but does not actually establish a connection.

- To establish a connection with the database, we need to call the `connect()` method using the `db` object, as shown:

```
JS index.js  X
JS index.js > ...
4
5  const db = mysql.createConnection({
6    host: 'localhost',
7    user: 'root',
8    password: 'root',
9    database: 'StudentCourseDB'
10  });
11
12  db.connect();
13
14  app.listen(5000, () => {
15    console.log('Server is running');
16  });
```

- To confirm whether a connection is established or if an error has occurred, update the `db.connect()` method to display an appropriate message using an `if` statement.

```
JS index.js > ...
8    password: 'root',
9    database: 'StudentCourseDB'
10  });
11
12  db.connect((err) => {
13    if (err) {
14      console.log('Error connecting to MySQL');
15      return;
16    }
17    console.log('Connected to MySQL');
18  });
19
```

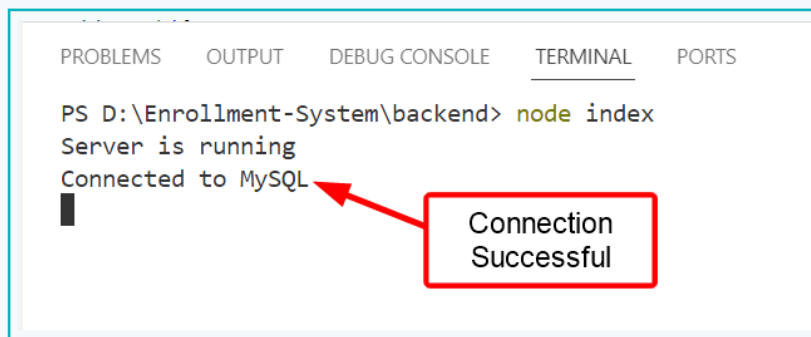
Here, if an error occurs, the `if` block will execute and display an error message. The `return` statement ensures that no further code runs in case of an error.

Activity

Note:

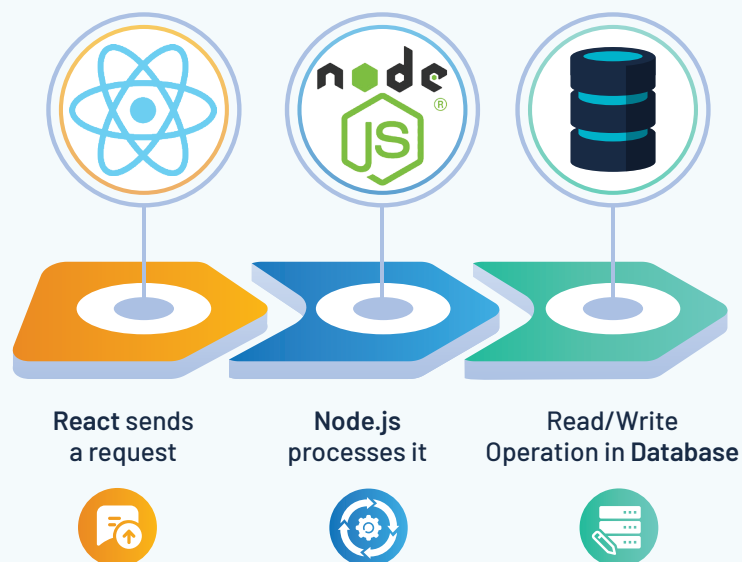
The 'err' object contains details of any error that occurs during the connection.

- Run the Node project using the `node index` command. If the database connection is successful, the "Connected to MySQL" message should be displayed.



```
PS D:\Enrollment-System\backend> node index
Server is running
Connected to MySQL
```

As already mentioned, in a full-stack web application, the frontend (React) cannot directly interact with the MySQL database. A backend API route is required to handle this, which accepts data from the frontend and inserts it into the database.



Now, let us define an API route in the Node.js project to insert student details into the 'Students' table.

Activity

Part 5 - Create Routes to Insert Student Details

- 1 In the 'index.js' file, define a route that uses the *POST* method and set the URL path to '/insert/student', as shown:

```
JS index.js > app.listen() callback
16 db.connect((err) => {
17   if (err) {
18     console.log('Error connecting to MySQL');
19     return;
20   }
21   console.log('Connected to MySQL');
22 });
23
24 app.post('/insert/student', (req, res) => {
25
26 });
27
```

- 2 Next, we need to extract the values that will be sent from the frontend (React).

```
19
20 app.post('/insert/student', (req, res) => {
21   const { std_name, std_email, std_grade, course_id } = req.body;
22 });
23
24 app.listen(5000, () => {
25   console.log('Server is running');
26 });
```

Here, *req.body* contains the data sent from the frontend, which is then stored in appropriate variables such as 'std_name', 'std_email', etc.

- 3 Next, we will define an *INSERT* query and store it in a variable 'insertStmt'. The four question marks (?) within *VALUES*() act as placeholders for the values that will be inserted.

```
19
20 app.post('/insert/student', (req, res) => {
21   const { std_name, std_email, std_grade, course_id } = req.body;
22
23   const insertStmt = "INSERT INTO students(std_name, std_email, std_grade, course_id) VALUES (?, ?, ?, ?)";
24
25 });
```

Activity

- 4 To execute the above SQL query, we need to use the `db.query()` function. This function requires the variable holding the SQL query, along with the values received from `req.body`. These values will replace placeholders (?) in the SQL query.

```
app.post('/insert/student', (req, res) => {  
  const { std_name, std_email, std_grade, course_id } = req.body;  
  const insertStmt = "INSERT INTO students(std_name, std_email, std_grade, course_id) VALUES (?, ?, ?, ?)";  
  db.query(insertStmt, [std_name, std_email, std_grade, course_id], (err) => {  
  })  
});  
app.listen(5000, () => {  
  console.log('Server is running');  
});
```

- 5 We will also define an `if` block to display a message in case of an error.

In case the query is executed successfully, send a response back to the client in JSON format using the `res.json()` method. This response will contain a `message` key along with a string value "Student Information Added".

```
app.post('/insert/student', (req, res) => {  
  const { std_name, std_email, std_grade, course_id } = req.body;  
  const insertStmt = "INSERT INTO students(std_name, std_email, std_grade, course.  
  db.query(insertStmt, [std_name, std_email, std_grade, course_id], (err) => {  
    if (err) {  
      console.log(err);  
    }  
    return res.json({ message: "Student Information Added" });  
  })  
});
```

This completes the creation of an API route for inserting records into the 'Students' table. According to this route, when a POST request is made to <http://localhost:5000/insert/student> with the required field values, the data will be inserted into the 'Students' table.

Before we proceed with making our frontend communicate with the backend, it is important to understand **CORS (Cross-Origin Resource Sharing)** and the `express.JSON()` middleware.

Activity

Part 6 - Implement CORS and EXPRESS.JSON()

About CORS

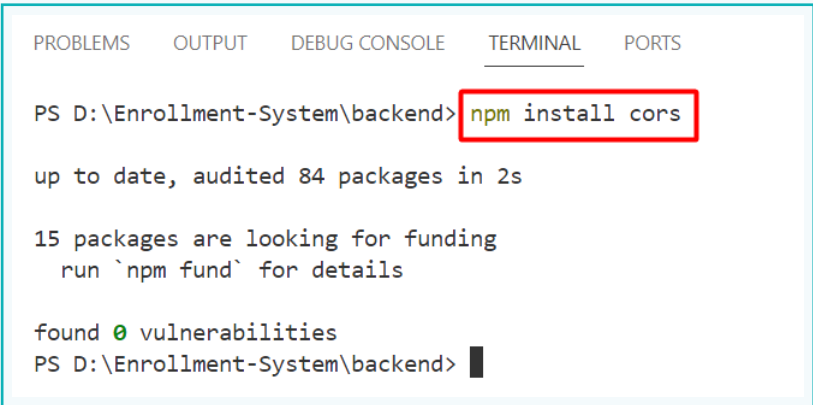
Browsers implement a certain security measure that prevents websites from making requests to other websites that are hosted on different servers or ports. This type of request, where a browser requests resources from a different origin than its own, is called Cross-Origin Request (CORS).

For example, a webpage on <https://example-1.com> requests an image or data from <http://example-2.com>

In our case, the frontend runs on <http://localhost:3000>, and the backend runs on <http://localhost:5000>. Since they are running on different port numbers, the browser will block any request coming from the frontend to the backend.

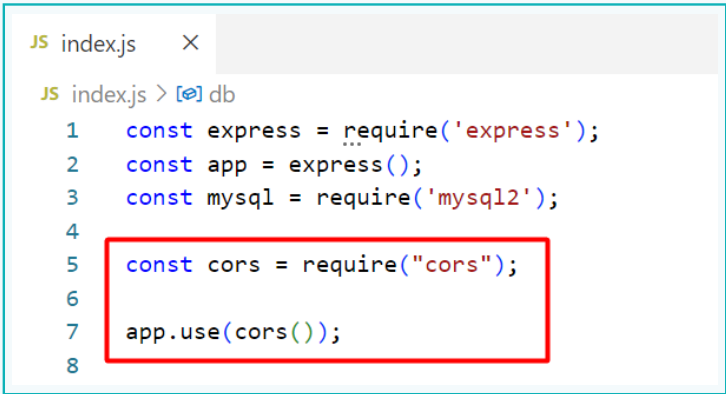
To resolve this issue, we need to configure CORS to allow cross-origin requests.

- 1 Install the *cors* library using the command `npm install cors`.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Enrollment-System\backend> npm install cors
up to date, audited 84 packages in 2s
15 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
PS D:\Enrollment-System\backend>
```

- 2 In the 'index.js' file, import *cors* and set it up using the `app.use(cors())` method.



```
JS index.js X
JS index.js > [db] db
1 const express = require('express');
2 const app = express();
3 const mysql = require('mysql2');
4
5 const cors = require("cors");
6
7 app.use(cors());
8
```

Activity

About express.JSON()

In a full-stack application, when the frontend (React) sends data to the backend, it is usually in **JSON (JavaScript Object Notation)** format. JSON is a text-based format for storing and transporting data, and Node.js does not automatically understand or process this format.

Additionally, the SQL queries we used earlier cannot work with JSON data directly and require a JavaScript object instead.

Therefore, the JSON data must be converted into a JavaScript object so that the Node.js application (Express) can easily process it.

The *express.json()* middleware automatically converts JSON data into a JavaScript object, making data handling simpler, safer, and more efficient.

- 3 In the 'index.js' file, set up the *express.json()* method as shown:

```
JS index.js > db.connect() callback
1  const express = require('express');
2  const app = express();
3  const mysql = require('mysql2');
4  const cors = require("cors");
5
6  app.use(cors());
7
8  app.use(express.json());
9
10 const db = mysql.createConnection({
11   host: 'localhost',
12   user: 'root',
13   password: 'root',
14   database: 'StudentCourseDB'
15 });
```

Cors and *express.json()* are now configured in our project. This ensures smooth communication between our **React** frontend and **Node.js + MySQL** backend, as well as handling and parsing of data sent from the frontend.

At this stage, our backend can connect to the database and retrieve data. To complete our full-stack application, we must link the React frontend with the Node.js backend. This will facilitate data exchange between the frontend and backend, allowing for database operations.

We will now update the Student Form (created in the previous lesson) in React to capture user input and send it to the backend.

Activity

Part 7 - Handle Form Inputs in React

Store Form Data in React State using React Hooks

React uses **state** to store dynamic values, such as user inputs. *State* is a built-in object in React that holds component-specific values. When the state changes, React re-renders the component to reflect the updated values.

To manage state, React provides **Hooks**, which allow components to handle dynamic data efficiently. The two most commonly used hooks are:

- ***useState hook***: Used to declare state variables and update their values.
- ***useEffect hook***: Tells React what a component should do after the first render and whenever the specified state variable changes. It is commonly used for tasks such as fetching data and making API calls.

These hooks play a crucial role in handling user interactions and managing data flow in a React application. Let us implement this and understand better.

Note:

The following operations should be implemented in the React application (frontend) we created in the previous lesson.

1 Declare State Variables.

The *useState* hook has the following syntax:

```
const [stateVariable, setStateFunction] = useState(initialValue);
```

- **stateVariable**: Holds the state value.
- **setStateFunction**: A function used to set or update the state value.
- **initialValue**: The starting value of the state.

Activity

- Open the 'StudentForm.js' file. Write the code to import the *useState* hook and declare four *states* ('std_name', 'std_email', 'std_grade', 'course_id') for storing the values of each form element.
- We also need to declare the *set** functions ('setStdName', 'setStdEmail', 'setStdGrade', 'setCourseID') for storing or updating values in the *states*.

```

1  import {useState} from 'react'
2
3  function StudentForm() {
4
5      const [std_name, setStdName] = useState('')
6      const [std_email, setStdEmail] = useState('')
7      const [std_grade, setStdGrade] = useState('')
8      const [course_id, setCourseID] = useState('')
9
10     return (
11         <div className='student-form'>
12             <h2> ADD STUDENT </h2>

```

Now, when a user types in the input fields. We need to invoke the *set** method to store the user inputs from the form and convert them into state values.

- In the input fields, use the *onChange()* event to set or update the state values based on the user's input.

```

16  return (
17      <div className='student-form'>
18          <h2> ADD STUDENT </h2>
19
20          <form>
21
22              <input type='text' placeholder='Enter Student Name' onChange={(e) => setStdName(e.target.value)} />
23
24              <input type='text' placeholder='Enter Student Email' onChange={(e) => setStdEmail(e.target.value)} />
25
26              <input type='text' placeholder='Enter Student Grade' onChange={(e) => setStdGrade(e.target.value)} />
27
28              <select onChange={(e) => setCourseID(e.target.value)}>
29
30                  <option value='' disabled selected> Select a Course </option>
31                  <option value='WEB-101'> Learning HTML and CSS </option>
32                  <option value='CS-102'> Starting with Computer Science </option>
33                  <option value='DS-103'> Data Science for Beginners </option>
34              </select>
35              <br />

```

Activity

Here,

- **onChange:** An event triggered every time a user types in the input field.
- **(e):** The event object containing details about the event, including the user's input values.
- **setStdName(), setStdEmail()...**: Functions to set or update value (from *useState*)
- **(e.target.value):** Refers to the current value typed in the input fields.

Once done with setting up the *change event* to capture and store user input in the state variables, we will implement the functionality to transfer the form data (from React) to the Node.js application and, finally, into the database.

Part 8 - Frontend to Backend Communication

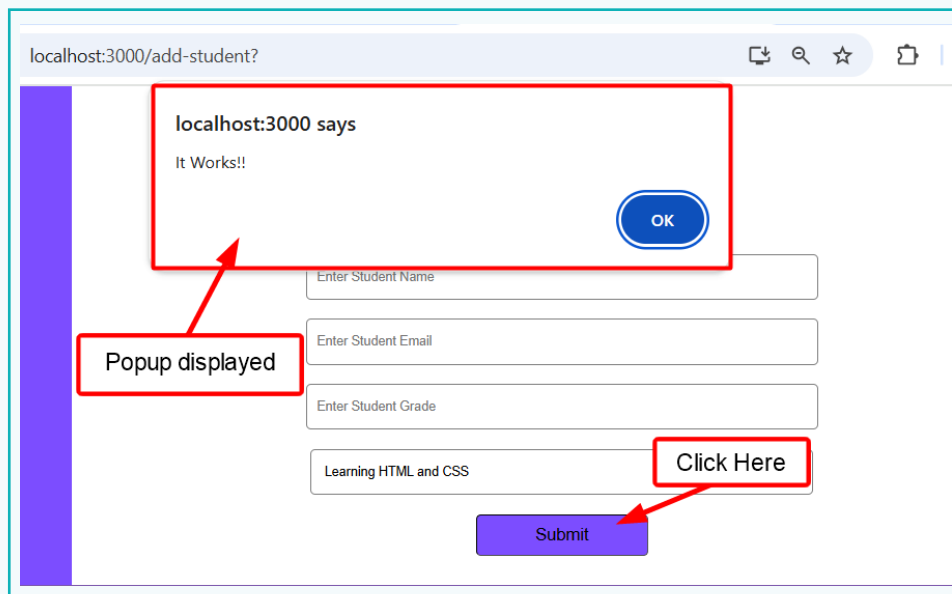
Here, we will learn how a React application can connect to a Node.js project, transfer form data, and handle the response from the backend.

- 1 Define a function *submitStudentData()* and define an *alert()* method inside it. Use the *onSubmit* handler to execute this function on form submission.

```
JS StudentForm.js ●
src > components > JS StudentForm.js > StudentForm
1  import {useState} from 'react'
2  function StudentForm() {
3      const [std_name, setStdName] = useState("")
4      const [std_email, setStdEmail] = useState("")
5      const [std_grade, setStdGrade] = useState("")
6      const [course_id, setCourseID] = useState("")
7
8      function submitStudentData(e) {
9          alert("It Works!!");
10     }
11     return (
12         <div className='student-form'>
13             <h2> ADD STUDENT </h2>
14
15             <form onSubmit={submitStudentData}>
16
```


Activity

- 2 Run the application to check whether the submit button is working.
 - In the 'Add Student' form, click on the 'Submit' button.
 - If an alert message with the text "It Works!!" appears, your code is correct.



React on its own cannot send requests to backend routes; we need libraries like **Axios** to perform this functionality. We will install the library first and import it.

- 3 Open the terminal and execute the command `npm install axios`.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Enrollment-System\frontend> npm install axios

up to date, audited 1352 packages in 5s

268 packages are looking for funding
  run `npm fund` for details
```

Activity

- 4 Import the *axios* library, and within the *submitStudentData()* function, use the *axios.post()* method to send a request to the backend (Node.js) server, available at “*http://localhost/5000*”.

```
JS StudentForm.js
src > components > JS StudentForm.js > StudentForm
1  import {useState} from 'react'
2  import axios from 'axios'
3
4  function StudentForm() {
5      const [std_name, setStdName] = useState("")
6      const [std_email, setStdEmail] = useState("")
7      const [std_grade, setStdGrade] = useState("")
8      const [course_id, setCourseID] = useState("")
9
10     function submitStudentData(e) {
11         axios.post("http://localhost:5000/insert/student",
12             {std_name, std_email, std_grade, course_id} )
13     }
14     return (
15         <div className='student-form'>
```

Here,

- “**http://localhost:5000/insert/student**”: Refers to the API route we defined in our Node.js application for inserting student details in the ‘Students’ table. *Refer to the ‘Part 5: Creating Routes to Insert Student Details’ above.*
- **{ std_name, std_email, std_grade, course_id }**: refers to the state variables containing the user input form values being sent along with the request.

- 5 To test whether it’s working, open the previously created ‘frontend’ folder in VS Code. Open the terminal and execute the ‘React Application’ using the *npm start* command.

```
file Edit Selection View Go Run Terminal Help
App.js - frontend - Visual Studio Code

EXPLORER
FRONTEND
  node_modules
  public
  src
    components
      Home.js
      Sidebar.js
      StudentForm.js
    App.css
    App.js
    App.test.js
    index.css
    index.js

src > JS App.js > ...
1  import './App.css';
2  import Home from './components/Home'
3  import Sidebar from './components/Sidebar'
4  import StudentForm from './components/StudentForm';
5  import { Routes, Route } from 'react-router-dom'
6
7  function App() {
8      return (
9          <div className="App">
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\Enrollment-System\frontend> npm start

Activity

- 6 Once the application runs in the browser, click on the 'Add Student' link in the sidebar. Enter data in the form fields and click on the 'Submit' button.

Click Here

Home

Add Student

Student Profiles

All Courses

Student Details

ADD STUDENT

Alex

Alex@gmail.com

10

Starting with Computer Science

Submit

- 7 To confirm whether the data is successfully stored in the 'Students' table, open MySQL client and execute the *SELECT* statement.

```
mysql> use StudentCourseDB;
Database changed
mysql>
mysql> SELECT * FROM STUDENTS;
+-----+-----+-----+-----+-----+
| std_id | std_name | std_email | std_grade | course_id |
+-----+-----+-----+-----+-----+
| 18 | Alex | Alex@gmail.com | 10 | CS-102 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Record Inserted

We can see that the data inserted in the frontend (React application) is successfully available in the backend (MySQL Database).

Note:

The value for the *std_id* field may be different in your table, as it is auto-generated.

For a better user experience, we can also display a confirmation message once a student record is successfully inserted into the database.

Activity

- 8 Using the `.then` handler, we will handle the response sent from the backend, as shown:

```

4  function StudentForm() {
5      const [std_name, setStdName] = useState("")
6      const [std_email, setStdEmail] = useState("")
7      const [std_grade, setStdGrade] = useState("")
8      const [course_id, setCourseID] = useState("")
9
10     function submitStudentData(e) {
11         axios.post("http://localhost:5000/insert/student",
12             { std_name, std_email, std_grade, course_id })
13
14         .then((response) => {
15             alert(response.data.message);
16             window.location.reload();
17         })
18     }
19 }

```

Here,

- **.then():** It is a function that automatically runs once the task is successfully completed. Here, we define what to do next once the response is received from the backend.
- **response:** Contains the result sent back to the frontend from the backend.
- **alert(response.data.message):** This refers to the *message* field that we are sending from the backend. Using `alert()`, we display the *message* field's value in a popup box.

```

JS index.js •
JS index.js > ...
25  app.post('/insert/student', (req, res) => {
26
27      const { std_name, std_email, std_grade, course_id } = req.body;
28      const insertStmt = "INSERT INTO students(std_name, std_email, std_
29
30      db.query(insertStmt, [std_name, std_email, std_grade, course_id],
31          if (err) {
32              console.log(err);
33          }
34          return res.json({ message: "Student Information Added" });
35      })
36  });
37
38

```

'message' field

(Refer to the backend route `/insert/student`)

- **window.location.reload():** This will automatically reload (refresh) the page, which is ideal for resetting the form.

Activity

- Insert another record using the form. Once you click on the 'Submit' button, a popup box appears. Click the 'OK' button to continue.

localhost:3000 says
Student Information Added

OK

Message Popup

Emma

emma123@gmail.com

7

Learning HTML and CSS

Submit

Click to submit

Part 9 - Create Routes to View Student Details

Let us define another route and associate functionality to view all the records stored in the 'Students' table.

- In your Node.js application, open the 'index.js' file and create a route `/view/students`. Inside this route, define a `SELECT` statement. Use `JOIN` to retrieve the values from both the 'Students' and 'Courses' tables. Store the query in a variable 'viewStmt'.

```

30 db.query(insertStmt, [std_name, std_email, std_grade, course_id], (err) => {
31   if (err) {
32     console.log(err);
33   }
34   return res.json({ message: "Student Information Added" });
35 }
36 );
37
38 app.get("/view/students", (req, res) => {
39   const viewStmt = 'SELECT * FROM Students s JOIN Courses c ON s.course_id = c.course_id';
40 });
41

```

Activity

- 2 Execute the query using the `db.query()` method. Define an `if` block to handle errors. In case the query is executed successfully, send a response back to the client (React project) using the `res.json(results)` method. Note that the results parameter contains the student's records retrieved from the database.

```

38 app.get("/view/students", (req, res) => {
39   const viewStmt = 'SELECT * FROM Students s JOIN Courses c
40
41   db.query(viewStmt, (err, results) => {
42     if (err) {
43       console.log(err)
44     }
45     return res.json(results);
46   });
47 });
48

```

Part 10 - Display Students Data in the Frontend

Recall:

We already created the 'StudentProfiles' component in the previous lesson. We will now update it by initiating an API call to the new route.

- 1 Open the 'StudentsProfile.js' file in VS Code and import `useState`, `useEffect` and `axios`. Also, declare a state variable 'students'.

```

EXPLORER
  FRONTEND
    node_modules
    public
    src
      components
        JS Home.js
        JS Sidebar.js
        JS StudentForm.js
        JS StudentsProfile.js
    App.css

src > components > JS StudentsProfile.js > StudentsProfile
1  import React, { useEffect, useState } from "react";
2  import axios from "axios";
3
4  function StudentsProfile() {
5
6    const [students, setStudents] = useState([]);
7
8    return (
9      <div className="data-view">
10     <h2> STUDENTS PROFILE </h2>

```

Activity

In `const [students, setStudents] = useState([])`,

- **students:** State variable to hold the list of students' data.
- **setStudents:** Function to update the state variable 'students'.
- **useState([]):** *useState* hook that initialises 'students' as an empty array.

Next, we need to make an axios call within the *useEffect()* React hook.

- 2 Create a *useEffect()* hook, as shown.

```
JS StudentsProfile.js
src > components > JS StudentsProfile.js > StudentsProfile
2 import axios from "axios";
3
4 function StudentsProfile() {
5   const [students, setStudents] = useState([]);
6
7   useEffect( () => {
8
9   }, []);
10
11   return (
12     <div className="data-view">
13       <h2> STUDENTS PROFILE </h2>
```

- 3 Make a *GET* request to the backend at "<http://localhost:5000/view/students>" route, using the *axios.get()* method.

- Inside *.then()*, update the state variable 'students' using *setStudents()*, with the data obtained from the backend (database).

```
JS StudentsProfile.js X
src > components > JS StudentsProfile.js > StudentsProfile
3
4 function StudentsProfile() {
5   const [students, setStudents] = useState([]);
6   useEffect( () => {
7
8     axios.get("http://localhost:5000/view/students")
9       .then((response) =>
10         setStudents(response.data)
11       )
12   }, []);
13
14   return (
15     <div className="data-view">
```

Now, we will display the data in the HTML table, which we created in the previous lesson (frontend).

Activity

- 4 Add the following code inside the empty `<tbody>` tag.

```
JS StudentsProfile.js x
src > components > JS StudentsProfile.js > StudentsProfile
21      <th> Name </th>
22      <th> Email </th>
23      <th> Grade </th>
24      <th> Course Enrolled </th>
25    </tr>
26  </thead>
27  <tbody>
28    { students.map((student) => (
29      <tr>
30
31      </tr>
32    )) }
33  </tbody>
34 </table>
35 </div>
```

Here,

- **`students.map( (student) => ( ... ) )`**: Here, we are using the `map()` method to loop over the data in the 'students' array (state variable). For each student object, we create a new table row (`<tr>`). For example, if three students are in the state variable, three table rows will be created, one for each student.

Now, we can display the data using the `<td>` tag.

- 5 Create table data (`<td>`) cells to display specific details of each student.

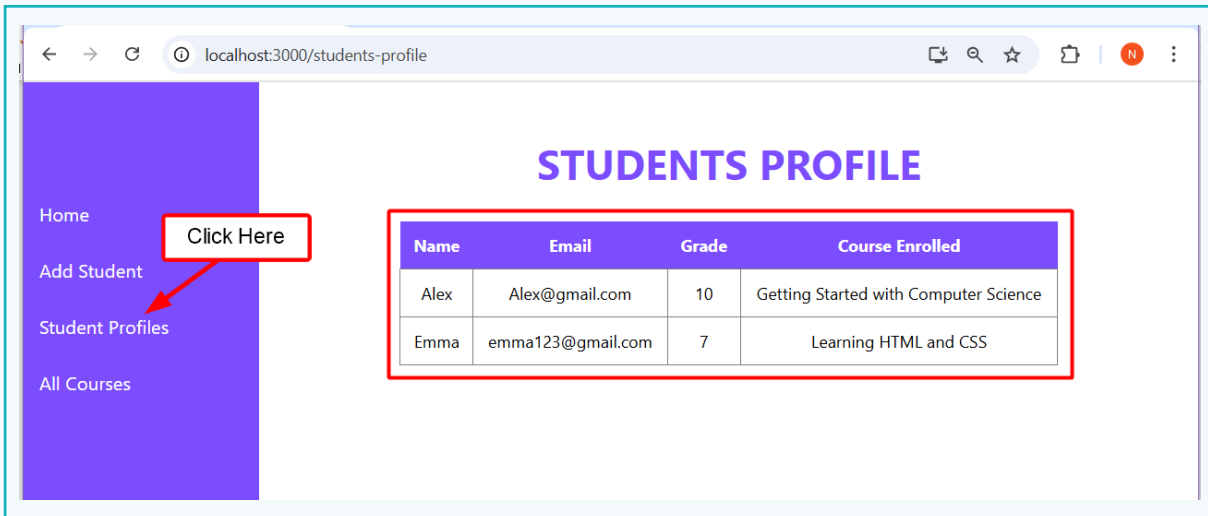
```
<tbody>
{ students.map((student) => (
  <tr>
    <td>{student.std_name} </td>
    <td>{student.std_email} </td>
    <td>{student.std_grade} </td>
    <td>{student.course_name} </td>
  </tr>
)) }
</tbody>
</table>
</div>
```

Here,

`{ student.std_name }` refers to the `std_name` field in the 'Students' table.

Activity

- 6 Execute the application and click on the 'Student Profiles' link in the Sidebar. A table displaying all the student records will appear.



From the output, we can observe that our full-stack application works as intended. You may add additional features to this application, such as displaying records from the 'Courses' table. Further, you may link it to the 'All Courses' item in the Sidebar.

In this lesson, we have successfully built a full-stack application where the 'React frontend' interacts with the 'Node.js backend' and 'MySQL database.' The system allows users to enter student details, store them in the database, and retrieve records dynamically, demonstrating seamless data flow between all components.

Exercise

Choose the Correct Answer.

- 1 To create a table in a database, which command is used?
 - a. GENERATE TABLE
 - b. CREATE TABLE
 - c. CREATE TABLES
 - d. MAKE TABLE
- 2 To automatically generate an incrementing value for a table field, which property is used?
 - a. AUTO INCREMENT
 - b. AUTO_INCREMENT
 - c. AUTO-INCREMENT
 - d. AUTOMATIC-INCREMENT
- 3 To install the 'express' framework, command used is.
 - a. npm install
 - b. node install
 - c. npm init
 - d. npm download
- 4 While defining a SQL query within a Node application, which symbol is used as a placeholder for the values to be inserted?
 - a. %
 - b. \$
 - c. ?
 - d. #

Fill in the Blanks.

- 5 To make a web server listen for incoming connections, the _____ method is used.
- 6 The _____ package is needed to connect a Node.js project with a MySQL Database.
- 7 CORS stands for _____.
- 8 The _____ hook is used to declare state variables.

Exercise

■ State if the Statements are True or False.

9 JSON stands for Java Object Notation.

▲ _____

10 UseEffect() is a type of React Hooks.

▲ _____

Tech Flash

MySQL	: A relational database Management System for managing a database.
CORS (Cross-Origin Resource Sharing)	: A security feature that allows or restricts web applications from making requests to a domain different from the one that served the web page.
JSON	: JSON (JavaScript Object Notation) is a text-based data format typically used for transmitting data between a server and a client in a readable format.
React Hooks	: Functions that allow you to use state and other React features in functional components.
Axios	: A JavaScript library to make HTTP requests from a web browser or Node.js application.